

Take-Home Project

RTX 5090 Decode Megakernel → Qwen3-TTS on Pipecat

Expected time: < 1 day with a good LLM coding agent

TL;DR

Take AlpinDale's **qwen_megakernel** (a ~1,200-line CUDA megakernel that runs Qwen3-0.6B at 1,000 tok/s on a single RTX 5090) and wire it up to serve **Qwen3-TTS** inference inside a **Pipecat** voice pipeline.

Reference Material

- Blog post: blog.alpindale.net/posts/5090_decode_optimization/
 - Source code: github.com/AlpinDale/qwen_megakernel
 - Pipecat docs: docs.pipecat.ai
 - Qwen3-TTS: huggingface.co/Qwen/Qwen3-TTS
-

What the Megakernel Does (Context)

- **Architecture:** 128 persistent thread blocks × 512 threads, launched as a single non-cooperative kernel
- **Model:** Qwen3-0.6B in bfloat16 (no quantization)
- **Performance:** ~1,000 tok/s decode (0.97 ms/step), 71% of theoretical GDDR7 bandwidth
- **Output:** single-token argmax per step (autoregressive decode loop on host)

The Task

Goal

Get the megakernel running as the LLM decode backend for Qwen3-TTS's **talker decoder** (not the codebook generator), **streaming real-time speech synthesis into a Pipecat voice agent pipeline**.

Performance Targets

- **TTFC (time to first audio chunk): < 60 ms**
- **RTF (real-time factor): < 0.15** — i.e. generating 1 second of audio must take less than 300 ms

- The output **must be streaming to Pipecat** — push audio chunks as they're decoded, do NOT buffer the full utterance before sending
- These are reference benchmarks for a good submission, not pass/fail cutoffs — but if you're way off, explain why

Step 1 — Adapt the Megakernel for Qwen3-TTS

- Clone github.com/AlpinDale/qwen_megakernel
- Qwen3-TTS uses the same Qwen3 architecture for its talker decoder stage — this is the target, not the codebook generator

Step 2 — Build the Inference Server

- Expose a simple streaming interface: prompt in → token stream out

Step 3 — Integrate with Pipecat

- Wire it into a basic Pipecat pipeline: STT → LLM → your TTS service → audio output

Step 4 — Validate End-to-End

- Run a round-trip test: speak → transcribe → LLM response → TTS → audio playback
- Measure and report: tokens/sec from the megakernel, TTFC (target < 50 ms), RTF (target < 0.1), overall latency
- Confirm audio is streaming to Pipecat frame-by-frame — not buffered-then-sent
- Confirm audio quality is acceptable (no glitches, dropped frames)

Deliverables

1. **Working repo** with build instructions (should work on a single RTX 5090)
2. **Short README** documenting: architecture decisions, any kernel modifications, how to run the Pipecat demo
3. **Performance numbers** — decode tok/s, TTFC (target < 90 ms), RTF (target < 0.3), end-to-end latency
4. **Demo recording** showing the voice agent working with you talking end to end

What We're Evaluating

- **Ramp-up speed** — how quickly and effectively you get up to speed and understand unfamiliar topics (CUDA kernels, TTS pipelines, Pipecat). We don't expect you to already know all of this
- **Performance rigor** — how thorough and honest your benchmarking and reporting is. Show us real numbers, methodology, and where the bottlenecks are. Don't hand-wave

- **Coding agent proficiency** — how extensively and effectively you leverage a modern LLM coding agent. We recommend Claude Code or Codex. **We will cover the costs** — don't hold back on usage
 - **Communication** — clear README, honest about what works and what's rough
-

Timing

With a good LLM coding agent (Claude Code, Cursor, etc.), this should take less than a day.

The megakernel already works. Qwen3-TTS is a known architecture. Pipecat has documented service interfaces. The work is integration, not research. If you're spending more than a day, you're probably overcomplicating it.

Notes

- You'll need an RTX 5090 (the kernel is tuned for sm_120 / Blackwell). We recommend renting one on [Vast.ai](#): — we will reimburse compute costs
- The megakernel is bfloat16 only — don't try to add quantization, that's not the point
- If the talker decoder's backbone is a different size than 0.6B, document what you changed in the kernel and why
- Bonus points: if you find a way to improve the megakernel's performance during integration, tell us about it